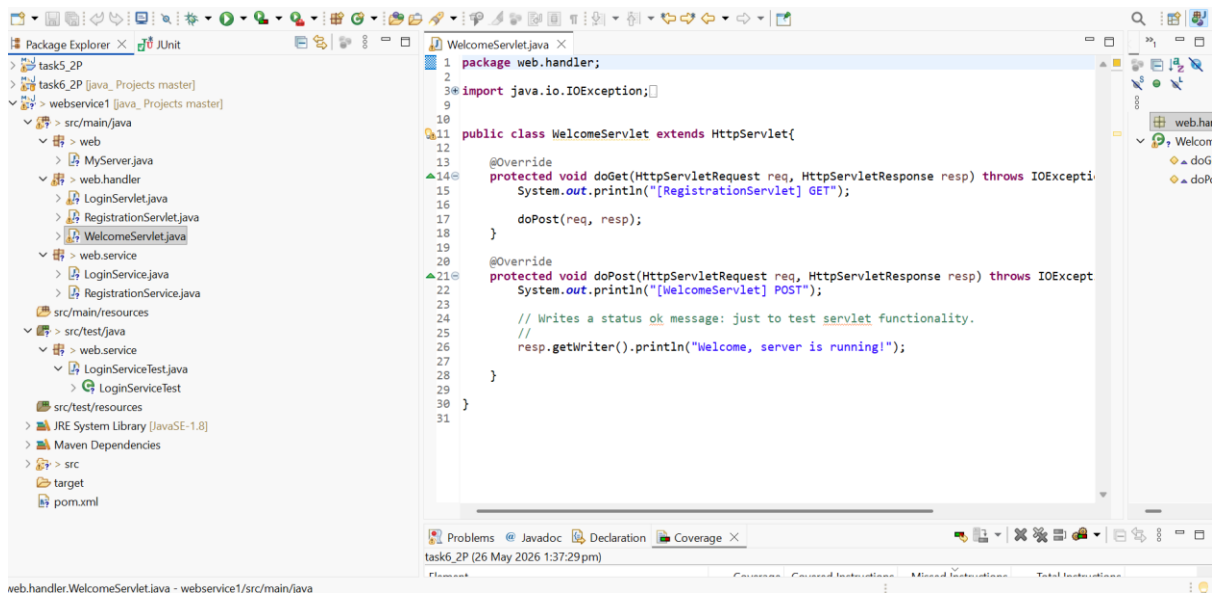
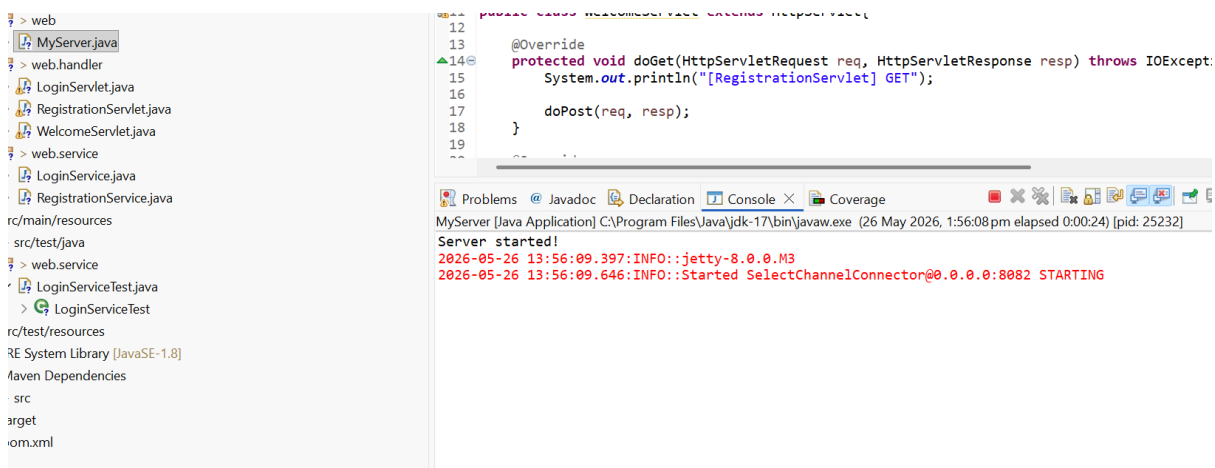


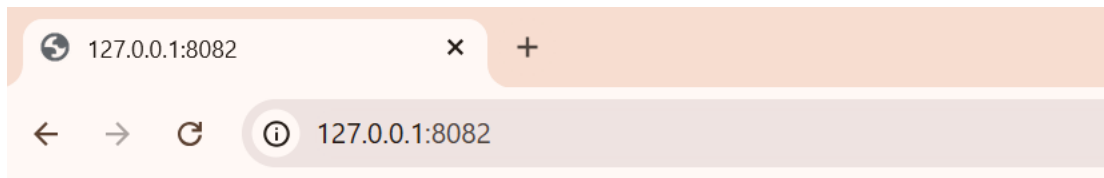
Pass Task 7: Integrate web front-end with Java API back end as part of integration testing



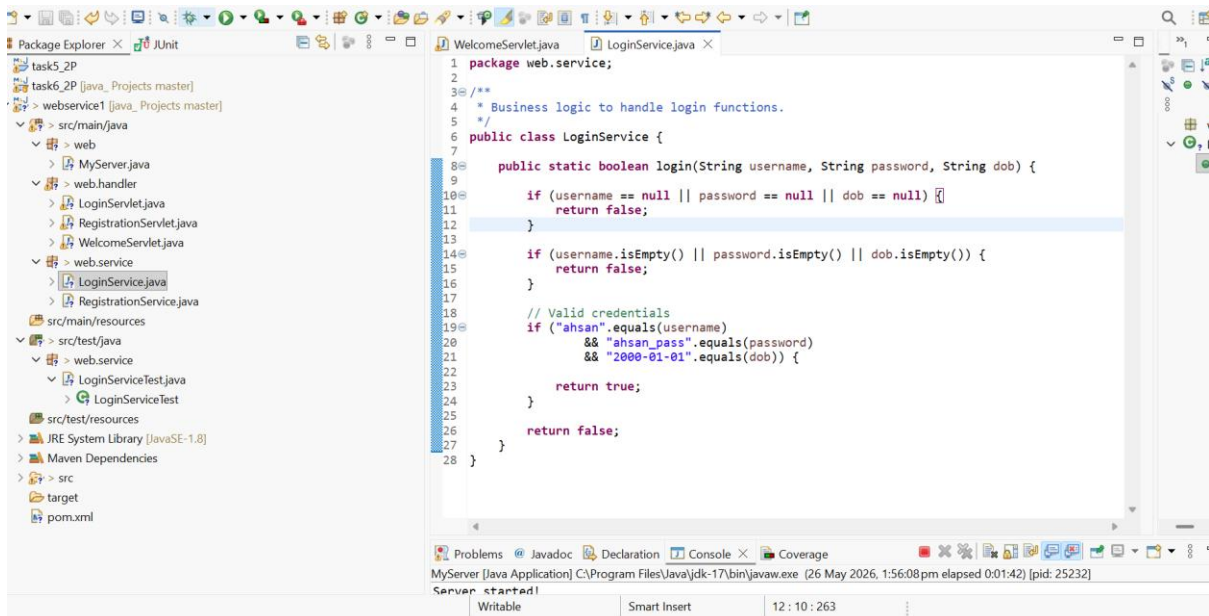
Imported 7.1p



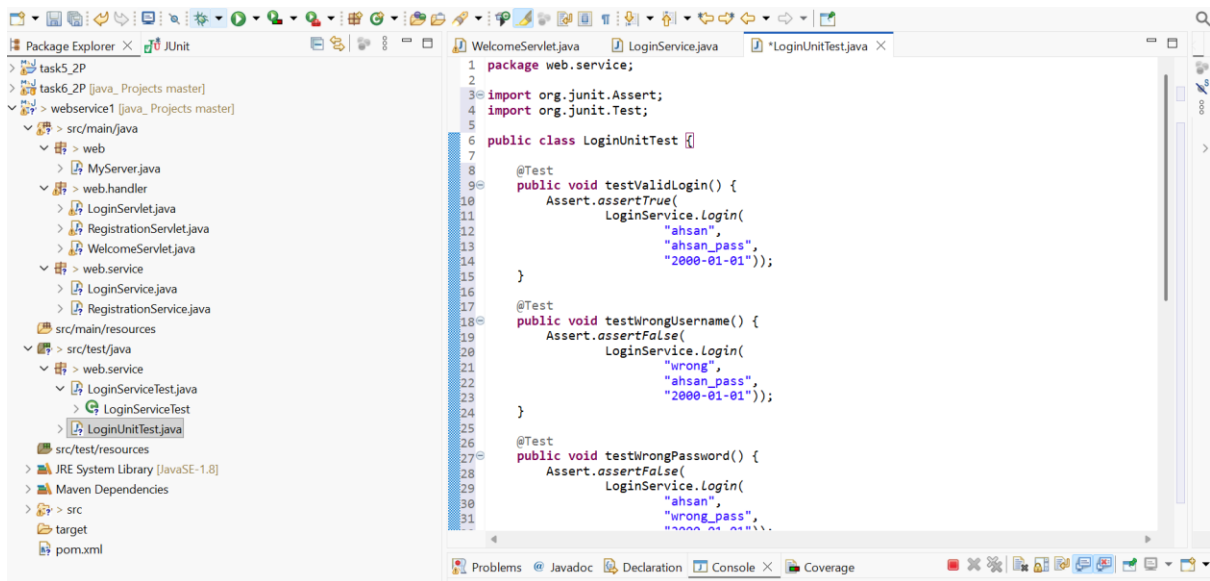
Server started



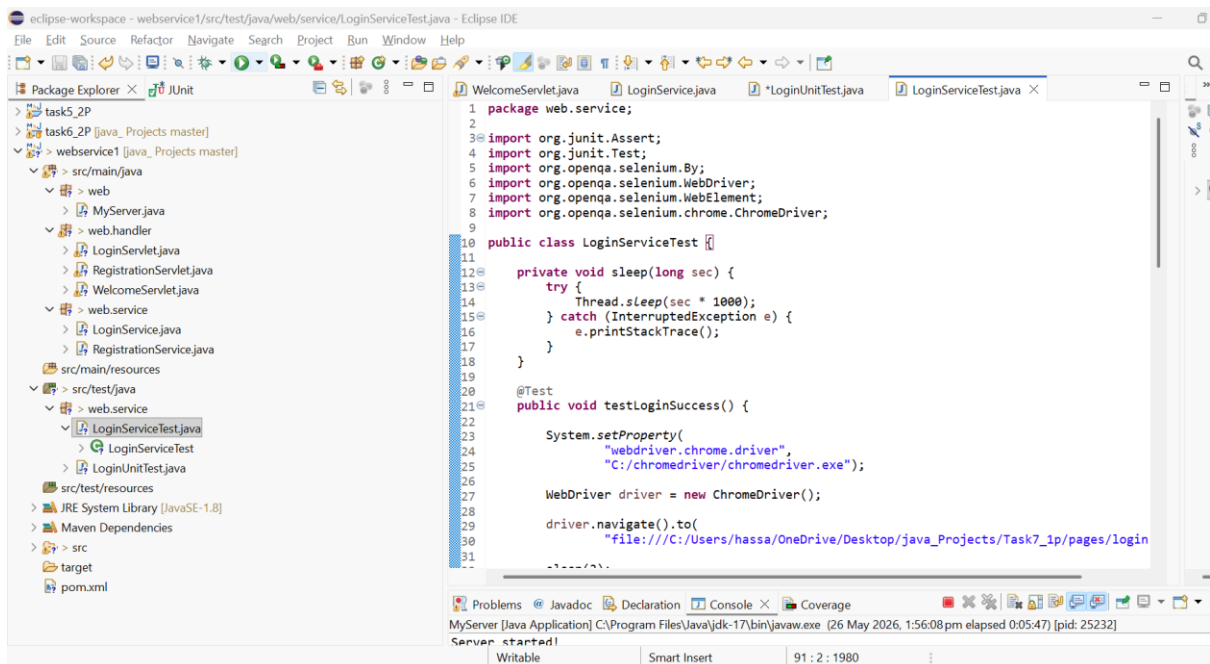
Welcome, server is running!



Updated loginservicejava file

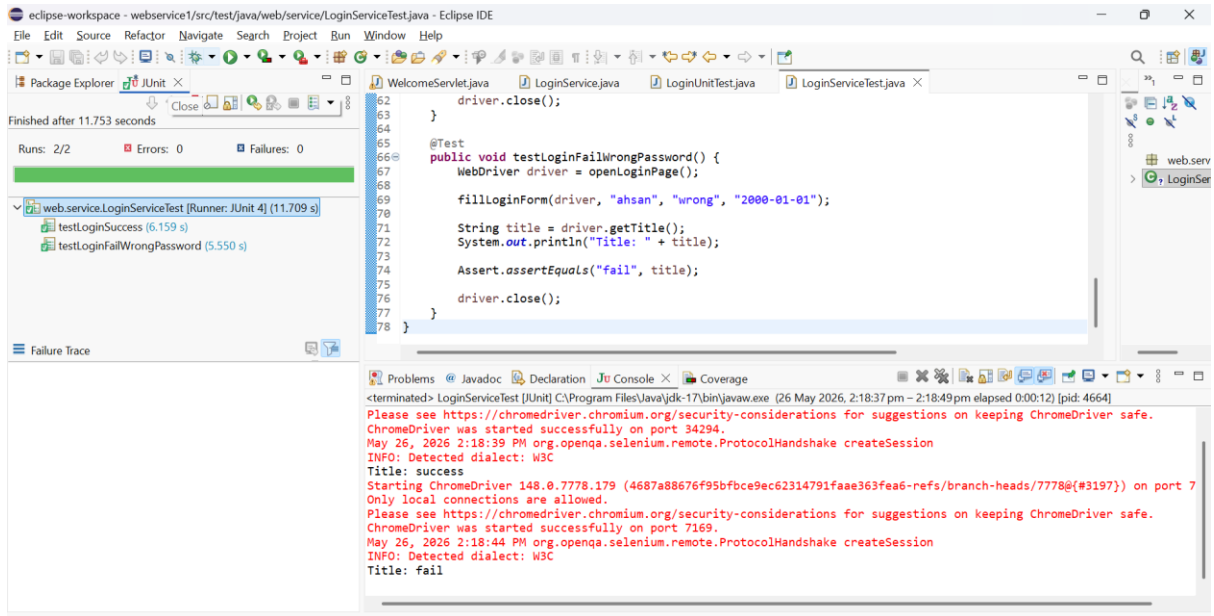


Created new loginunittest java file

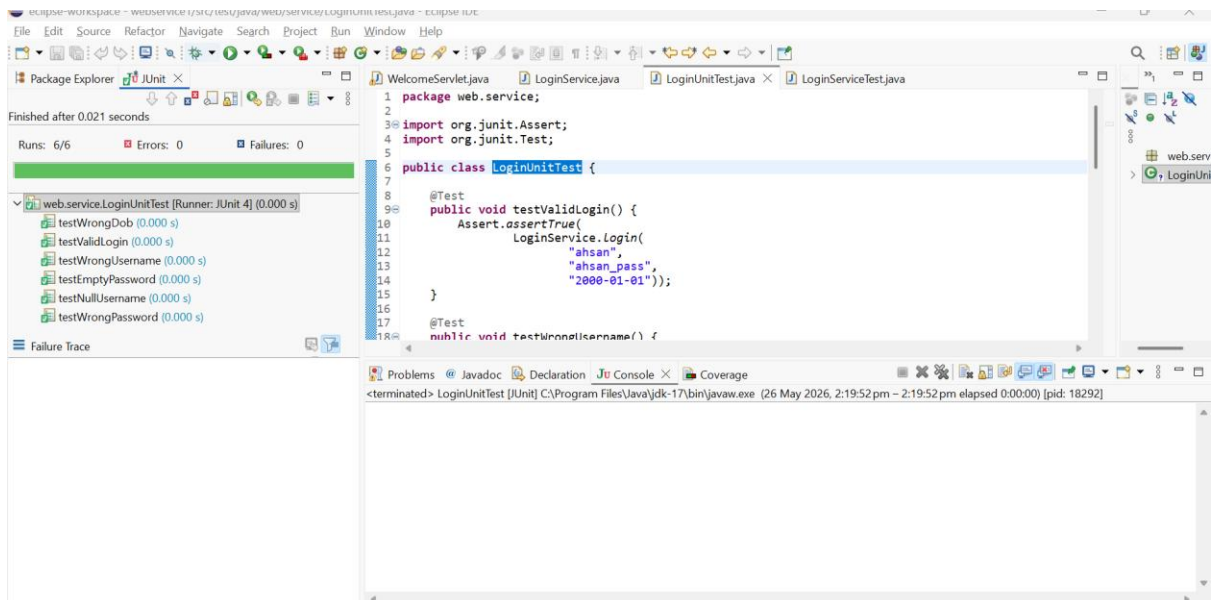


Loginservertest Updated Fixed Selenium Test

Installed Chromedriver



Also ran loginunittest works properly



Ran as coverage

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
web.service1	46.9 %	219	248	467
src/main/java	12.5 %	35	245	280
web.handler	0.0 %	0	158	158
web.service	44.3 %	35	44	79
RegistrationService.java	0.0 %	0	41	41
LoginService.java	92.1 %	35	3	38
web	0.0 %	0	43	43
src/test/java	98.4 %	184	3	187
web.service	98.4 %	184	3	187
LoginServiceTest.java	98.0 %	145	3	148
LoginServiceTest	98.0 %	145	3	148
LoginUnitTest.java	100.0 %	39	0	39
LoginUnitTest	100.0 %	39	0	39

Self-Reflection and Discussion

This task helped me understand the difference between unit testing and functional testing in a real web application environment. Unit testing focused on testing the internal logic of the `LoginService.java` file by checking different input combinations such as valid username, password, and date of birth values. These tests were written from the developer's point of view to verify that the authentication logic worked correctly and handled invalid or empty inputs properly.

Functional testing using Selenium focused on testing the system from the user's perspective. Selenium automated the browser, entered values into the login form, clicked the submit button, and checked whether the returned page title was success or fail. This confirmed that the front-end HTML form and the back-end Java service were correctly integrated.

The project also demonstrated aspects of integration testing because the tests verified communication between the HTML interface, servlet handlers, and authentication service. One limitation is that functional testing may not cover all internal logic paths compared to unit testing. Combining both testing approaches improved overall reliability and helped achieve high code coverage while validating both system behaviour and user interaction.

Source Code



LoginService.java



LoginServiceTest.java



LoginUnitTest.java

Github Push

github.com/IssacJ1/IssacSIT707

IssacJ1 / IssacSIT707

Type ↵ to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

IssacSIT707

Public

Pin

Watch 0

Fork 0

Star 0

main 1 Branch 0 Tags

Go to file

Add file

Code

IssacJ1 Added task6_1P code coverage task

b7c9d3a · now 9 Commits

Task_4.1_p

first commit of task4.1

last month

task2_1P

task_2.1

2 months ago

task3_1P

first commit

last month

task5_1P

Added task5_1P Maven project

1 hour ago

task6_1P

Added task6_1P code coverage task

now

week01

week01 task_1.2

2 months ago

.gitignore

task_2.1

2 months ago

New Text Document.txt

Added task5_1P Maven project

1 hour ago

welcome.md

Add welcome message to welcome.md

2 months ago

README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

IssacJ1